

Anomaly Detection in Syslog Streams: A Comparative Evaluation of Linear and Deep Learning Architectures

Capstone Review 1 Document

Team Members:

- Venkata Dhanush Kakarlamudi (23BCE8172)
 - Veera Nidhish Gondimalla (23BCE7421)
 - S Raja Rushi Praneeth Reddy (23BCE7425)
 - E Venakata Lakshmi Veereswar (23BCE7439)
-

1. Abstract

The goal of this project is to develop and evaluate robust architectures for log anomaly detection across different log datasets. Initial phases of the project involved replicating standard frequency-based linear models such as Principal Component Analysis (PCA) and Robust Principal Component Analysis (RPCA) on structured logs from the Hadoop Distributed File System (HDFS) and complex, real-world interleaved logs from the Blue Gene/L (BGL) supercomputer. The results established the brittleness of traditional count-based models on complex data. Subsequently, the project explored Semantic Natural Language Processing (NLP) embeddings (Word2Vec) and Deep Learning Autoencoders to capture contextual execution paths, significantly improving detection performance. This report outlines the progression of these methodologies and experiments conducted up to Review 1.

2. Introduction & Problem Statement

System logs are a critical data source for monitoring the health and security of large-scale distributed systems. However, detecting anomalies in these logs is challenging due to the massive volume of data, the unstructured nature of log messages, and the complex interleaving of processes.

Traditional approaches often rely on count-based matrices (term-frequency) of discrete log events. While these work well for highly structured, low-rank data, they fail to capture the sequential and semantic context of log execution paths. Our problem statement focuses on overcoming the limitations of these count-based linear models when applied to complex, high-variance datasets like BGL, where simple event frequencies are insufficient to establish a distinct normal operational subspace.

3. Progress Till Review 1: Methodology & Experiments

Our workflow till Review 1 has been broken down into five major experimental phases to iteratively diagnose limitations and improve the architecture.

Phase 1: Baseline Replication (Count-Based Linear Models)

- **Method:** We replicated standard baseline methods constructing a term-frequency matrix of discrete log events and applied linear projection techniques (PCA, RPCA).

- **HDFS Dataset:** PCA achieved an F1-score of 94.38%, cleanly matching expected targets on highly structured, low-rank data. However, RPCA suffered a severe "rank-collapse," yielding only 53.96% F1.
- **BGL Dataset:** When tested on the strict, complex BGL dataset, the PCA baseline collapsed entirely to **30.18% F1**. Even with an oracle threshold sweep, the model reached a strict theoretical ceiling of ~65% F1, proving that the linear approach is fundamentally inadequate for highly interleaved logs.

Phase 2: Sequence Order Embeddings (Semantic NLP)

- **Method:** To solve the rank-collapse and capture sequence meaning, we replaced sparse count matrices with dense, continuous semantic vectors (Word2Vec).
- **Results:** On HDFS, the dense vectors resolved the rank-collapse, allowing RPCA to recover to 84.96% F1. On BGL, however, performance initially degraded due to the temporal extraction of logs creating a "semantic soup" of interleaved server node outputs.

Phase 3: Structural Node-Based Grouping

- **Method:** We corrected the BGL log extraction process. Instead of grouping arbitrarily by time, we grouped logs logically by their originating **Node** identifier and a 6-hour sliding **Window_ID**.
- **Results:** This aligned the sequences with true physical execution paths. PCA performance on these structurally grouped embeddings improved significantly to **75.00% F1**, bypassing the previous 65% ceiling and proving the value of semantic transitions.

Phase 4: Deep Learning Autoencoder on Semantics

- **Method:** Linear projections (PCA/RPCA) are sub-optimal for non-linear continuous embedding spaces. We replaced them with a multi-layer perceptron (MLP) Autoencoder designed to reconstruct the 32-dimensional semantic vectors.
- **Results:** Under a rigorous 5-fold cross-validation on over 1.15 million sequences, the Autoencoder achieved an average **87.46% F1-score**.

Phase 5: Re-Evaluating the Baseline with Dynamic Peaks-Over-Threshold (POT)

- **Method:** We re-evaluated the hardcoded 95th-percentile threshold from standard literature, discovering a "tie-inflation" bug in standard rank-based error metrics. To fix this, we implemented dynamic Extreme Value Theory via Peaks-Over-Threshold (POT).
- **Results:** Our dynamic POT threshold raised the baseline performance on HDFS to an impressive **95.95% F1**, actively modeling the tail distribution rather than forcing a strict percentage cutoff.

4. Conclusion of Phase 1

The experiments up to this point have definitively shown that standard frequency-count linear models are brittle and fail on complex, interleaved system logs. Shifting to semantic NLP embeddings paired with Deep Learning (Autoencoders) allows the system to understand the actual contextual flow of logs, leading to highly robust anomaly detection. However, the model still shows sensitivity to threshold calibration and potential data distribution shifts, which will need to be addressed in subsequent phases.

5. Future Scope & Roadmap (For Review 2 & 3)

As we progress toward the next review cycles, the architecture will remain flexible for significant strategic pivots. Planned explorations include:

1. **Advanced Sequence Modeling:** Investigating whether deeper sequence models (like Transformers, Long Short-Term Memory networks (LSTMs), or Small Language Models (SLMs)) can capture long-range dependencies better than simple Word2Vec aggregations.
2. **Addressing Data Leakage & Feature Engineering:** Ensuring strict isolation between training embeddings and evaluation data to guarantee robust out-of-sample performance.
3. **Alternative Architectures:** Remaining open to entirely new paradigms, such as contrastive learning or self-supervised anomaly detection, should the Autoencoder hit a performance plateau.
4. **Real-Time Feasibility:** Evaluating the inference time of the chosen deep learning architectures for live log streams.